```
#include <iostream>

using namespace std;

void aliases(){

// an alias more commonly known as reference (l-value reference) is simply second name of a variable.

// when an alias is created compiler adds a second entry in its symbol table for the same variable.

// while defining aliases its type must match with the type of variable, only exceptions are const references.

// aliases must be initialized.

   int a=10, b=20;

   int &aliasi = a;

   cout<<"\n address of variable a = "<<&a

   <<"\n size of alias = "<<sizeof(aliasi)<<", address = "<<&aliasi<<", value in alias = "<<aliasi<<endl;

   aliasi = b;

   cout<<"\n address of variable b = "<<&b<<", value of b ="<<b

   <<"\n size of alias = "<<sizeof(aliasi)<<", address = "<<&aliasi<<", value in alias = "<<aliasi

   <<endl;

   // float &aliasf = a;

   //const float &aliacef = a;

}

void sizeOfPointers(){

   //A pointer can only store addresses.

   //A pointer itself does not have a data type but points to a data type.

   //Pointer stores the address of the first byte of the data type to which it points.

   //syntax: data-type-pointed-to dereferencing-operator pointer-variable-name;

   //dereference operator is also called indirection operator and is represented by the symbol *

   //size of pointers depends on the architecture.

   //on a 64bit (8 byte) architecture, pointer size = 8 bytes.

   //on a 32bit (4 byte) architecture, pointer size = 4 bytes.
```

```
   //declaring pointer, wild pointers (not initialized).
   int* iptr;        //pointer to integer variable
   float* fptr;      //pointer to float variable
   char* cptr;       //pointer to character variable
   double* dptr;     //pointer to double variable
   long long* lptr;  //pointer to long long variable
   long double* ldptr; //pointer to long double variable
   void* vptr1;      //pointer to void: can store addresses, but not associated with any data type.
   const int* cniptr = nullptr;  //pointer to constant integer
   cout<<"\n sizes: \n"<<"int pointer size ="<<sizeof(iptr)<<endl
   <<"float pointer size ="<<sizeof(fptr)<<endl
   <<"char pointer size ="<<sizeof(cptr)<<endl
   <<"double pointer size ="<<sizeof(dptr)<<endl
   <<"long long pointer size ="<<sizeof(lptr)<<endl
   <<"long double pointer size ="<<sizeof(ldptr)<<endl
   <<"void pointer size ="<<sizeof(vptr1)<<endl
   <<"pointer to constant integer size ="<<sizeof(cniptr)<<endl;
}
void assigningValuesAndDeReferencing( ){
   //assigning values to pointers
   //pointers can only store memory addresses.
   //pointer is assigned (stores) address using & (address-of) operator.
   //pointer stores the address of first byte of the memory assigned to it.
   /*
    pointer points to a data type: knows about number of bytes the original
    value resides in and the type of value.
   */
   //to access a value stored in pointer * (dereference operator) is used.
   /*
```

```
when a value is dereferenced using * operator it gets the value starting
from first byte up to the number of bytes allocated through data type
Pointer gets the information about total bytes through the data type pointed
to by the pointer.
*/
//declaring variables.
int i=1; float f=2.0; char c='a'; double d=3.34567; long long l=4;
long double ld = 5.123456789; const int cni = 786;
//declaring pointers.
int* iptr; float* fptr;
char* cptr;
//a pointer to character when printed with cout<< will be treated as a string.
double* dptr; long long* lptr; long double* ldptr; void* vptr;
const int* cniptr = nullptr;   //pointer to constant integer
int* const icnptr = &i;    //constant pointer to integer variable
const int* const cicnptr = &cni; //constant pointer to constant integer
iptr = &i;
fptr = &f;
cptr = &c;
dptr = &d;
lptr = &l;
ldptr = &ld;
cniptr = &cni;
cout<<"\n address of int variable = "<<iptr<<", value = "<<*iptr
<<"\n address of float variable = "<<fptr<<", value = "<<*fptr
<<"\n address of char variable = "<<cptr<<", value = "<<*cptr
<<"\n printing address of char variable through & operator = "
<<&c
//<<"\n printing address of char variable through casting ="
```

```
    //<<static_cast<void*>(cptr)
    <<"\n address of double variable = "<<dptr<<", value = "<<*dptr
    <<"\n address of long long variable = "<<lptr<<", value = "<<*lptr
    <<"\n address of long double variable = "<<ldptr<<", value = "<<*ldptr
    <<"\n address of constant integer in pointer variable = "
    <<cniptr<<", value = "<<*cniptr
    <<"\n address of integer variable in constant pointer= "
    <<icnptr<<", value = "<<*icnptr
    <<"\n address of constant integer in constant pointer= "
    <<icnptr<<", value = "<<*icnptr
    <<endl;
}
void voidPointers(){
    //void pointers
    //pointer to void: can store addresses, but not associated with any data type.
    /*
     since void pointers have no information about data type. they cannot
     dereference a value.
    */
    /*
     to dereference a value through void pointer it must be casted to appropriate
     data type.
    */
    int i=1; float f=2.0; char c='a'; double d=3.34567;
    long long l=4; long double ld = 5.123456789; const int cni = 786;
    void* vptr = nullptr;
    const void* cndvptr;
    cout<<"\n using void pointers \n";
    vptr = &i;
```

```
   cout<<"\n address of int variable = "<<vptr
   <<", value ="<<*( static_cast<int*>(vptr) );
   vptr = &f;
   cout<<"\n address of float variable = "<<vptr
   <<", value ="<<*( static_cast<float*>(vptr) );
   vptr = &c;
   cout<<"\n address of char variable = "<<vptr
   <<", value ="<<*( static_cast<char*>(vptr) );
   vptr = &d;
   cout<<"\n address of double variable = "<<vptr
   <<", value ="<<*( static_cast<double*>(vptr) );
   vptr = &l;
   cout<<"\n address of long long variable = "<<vptr
   <<", value ="<<*( static_cast<long long*>(vptr) );
   vptr = &ld;
   cout<<"\n address of long double variable = "<<vptr
   <<", value ="<<*( static_cast<long double*>(vptr) );
   cndvptr = &cni;
   cout<<"\n address of constant integer = "<<cndvptr
   <<", value ="<<*( static_cast<const int*>(cndvptr) );
   cout<<endl;
}
pointersAndConstants( ){
   //constants must be initialized.
   const float pi = 22.0/7;
   float f =2.234;
   const float* cnfptr = &f;        //pointer to constant float
   float* const fcnptr = &f;        //constant pointer to float variable
   const float* const cnfcnptr = &pi;  //constant pointer to constant float
```

```
   cout<<"\n constants & pointers ";
   cout<<"\n address of constant float in pointer variable = "<<cnfptr
   <<", value pointed to by pointer variable = "<<*cnfptr<<endl;
   cout<<"\n address of float variable in constant pointer = "<<fcnptr
   <<", value pointed to by pointer variable = "<<*fcnptr<<endl;
   cout<<"\n address of constant float in constant pointer = "<<cnfcnptr
   <<", value pointed to by constant pointer = "<<*cnfcnptr<<endl;
   //cnfptr = nullptr;
   //fcnptr = nullptr;
   //cnfcnptr = nullptr;
}
void PointerArithmetics_Arrays_PointerDecay( ){
   //Pointer arithmetics & relationship with arrays.
   //Whenever an arithmetic operations is applied on pinter, it multiplies the number with
sizeof(data type).
   //For example:  ptr + 1  = ptr + 1 * sizeof (dataType pointed to by ptr)
   //An array name behaves like a constant pointer by default.
   //if the array name is directly accessed it goes through "pointer decay".
   //Pointer decay: when size of the pointer is reduced to a single element of the pinter.
   /*
    Array name does not go through pinter decay:
      1. if the address operator is applied to the pointer
      2. when it is passed to the function by reference.
   */
   int arrP[4] = {2,4,6,8}; // arrP is a pointer and it points to:  int (*)[4]
   cout<<"\n address of first location through & = "<<&arrP;
   cout<<"\n address of first location in arrP = "<<arrP;
   cout<<"\n size of arrP = "<<sizeof(arrP);
   cout<<"\n size of &arrP (meaning size of pointer) = "<<sizeof(&arrP);
```

```
   cout<<"\n value at first location = "<<*arrP;
   cout<<"\n address of second location in arrP = "<<arrP + 1;
   cout<<"\n value at second location in arrP = "<<*(arrP + 1);
   cout<<"\n address of third location in arrP = "<<arrP + 2;
   cout<<"\n value at third location in arrP = "<<*(arrP + 2);
   cout<<"\n address of fourth location in arrP = "<<arrP + 3;
   cout<<"\n value at fourth location in arrP = "<<*(arrP + 3);
   // garbage values. why?
   cout<<"\n address of fifth location in arrP = "<<arrP + 4;
   cout<<"\n value at fifth location in arrP = "<<*(arrP + 4);
   cout<<endl<<endl;
   // through loop.
   cout<<"printing through loop"<<endl;
   for (int i=0; i<4; i++){
      cout<<"address of location "<<i<<" = "<<(arrP+i)<<endl;
      cout<<"value at location "<<i<<" = "<<*(arrP+i)<<endl;
   }
/*
int a1[4] = {0}, a2[4] = {1,2,3,4};
a1 = a2;
*/
}
void pointerToEntire1DArray(){
 int a[4]={1,2,3,4}, b[5]={11,12,13,14,15};
 // Pointer to 1D Array.
 int (*aptr)[4];
  /*
    syntax: data type (*pointer name)[n] creates A pointer to entire array of size n.
```

when creating a pointer to entire array parenthesis must be put around * and pinter name, i.e. ( * pointer name ).

this pointer can only store address of single array but complete array.

think of the pinter array like this: it will always create an additional dimension for the array such that:

1. the first dimension has a single location pointing to array whose address will be assigned to the pointer array.

2. second dimension will have n locations (4 in this case), each location pointing to a sing element of the array.

*/

```
 aptr=&a;
 cout<<"size of pointer array = "<<sizeof(aptr);
 cout<<"\n size of first location in pointer array = "<<sizeof(aptr[0]);
 cout<<"\n size of second location in pointer array (invalid dimension) = "<<sizeof(aptr[1]);
 cout<<"\n size of third location in pointer array  (invalid dimension) = "<<sizeof(aptr[2]);
 cout<<"\n size of fourth location in pointer array  (invalid dimension)= "<<sizeof(aptr[3]);
 cout<<"\n address of a[0] in array a = "<<&a[0];
 cout<<"\n address of a[0] through pointer array = "<< &aptr[0][0];
 cout<<"\n address of a[1] in array a = "<<&a[1];
 cout<<"\n address of a[1] through pointer array = "<< &aptr[0][1];
 // aptr=&b;
 cout<<"\n printing contents of array through pointer to entire array = ";
 for(int i=0; i<sizeof(a)/sizeof(a[0]); i++)
   cout<<(*aptr)[i]<<"\t";
 cout<<"\n printing contents of array through pointer to entire array using dimension concept = ";
 for(int i=0; i<sizeof(aptr[0])/sizeof(aptr[0][0]); i++)
   cout<<aptr[0][i]<<"\t";
}
void pointerToEntire2DArray(){
   int two[2][2] = { {21,22},{23,24}};
```

```
   // pointer to an entire two dimensional array.
   int (*twoptr)[2][2] = &two;
   cout<<"\n data at first location in array two = "<<twoptr[0][0][0];
   cout<<"\n data at second location in array two = "<<twoptr[0][0][1];
   cout<<"\n data at third location in array two = "<<twoptr[0][1][0];
   cout<<"\n data at fourth location in array two = "<<twoptr[0][1][1];
   cout<<"\n \n printing data in two dimensional array through pointer = ";
   for (int i=0; i<sizeof(twoptr[0])/sizeof(twoptr[0][0]); i++)
      for(int j=0; j<sizeof(twoptr[0][0])/sizeof(twoptr[0][0][0]); j++)
         cout<<twoptr[0][i][j]<<"\t";
}
void pointerToEntire3DArray(){
   int threed[3][2][1] = { { {31},{32}},{{33},{34}},{{35},{36} } };
   // pinter to an entire three dimensional array.
   int (*threedptr)[3][2][1] = &threed;
      cout<<"\n \n printing data in three dimensional array through pointer = ";
      for (int i=0; i<sizeof(threedptr[0])/sizeof(threedptr[0][0]); i++)
       for (int j=0; j<sizeof(threedptr[0][0])/sizeof(threedptr[0][0][0]); j++)
        for (int k=0; k<sizeof(threedptr[0][0][0])/sizeof(threedptr[0][0][0][0]); k++)
          cout<<threedptr[0][i][j][k]<<"\t";
}
void AccessingArrayWithoutDecay( ){
   int arrP[4] = {2,4,6,8}; //arrP pints to int (*)[4]
   //important point: data_type (*ptr)[n] creates A pointer to entire array of size n.
   int (*arrP_Ptr)[4] = &arrP; // applying address-of (&) operator to array name removes decay
operation.
   cout<<"address of arrP[0] through array name = "<<arrP;
   cout<<"\n address of arrP[0] through & = "<<&arrP;
   cout<<"\n address of arrP[0] through &arrP[0] = "<<&arrP[0];
```

```
   cout<<"\n address of arrP[1] through &arrP + 1 (outside loop) = "
      <<&arrP + 1; //moves outside the array block, proves no decay occurs when & applied.
   for (int i=0; i<4; i++)
      cout<<"\n address of iarr["<<i<<"] = "<< &(*arrP_Ptr)[i]
      <<", value at iarr["<<i<<"] = "<< (*arrP_Ptr)[i];
   int *ptr1 = arrP;
//    int *ptr2 = &arrP;
   cout<<endl;
}
AccessingArrayThroughPointerDecay(){
   int iarr[5] = {10, 20, 30, 40, 50};
   int *iaptr = iarr; // Points to the first element: pointer decay
   for (int i=0; i<sizeof(iarr)/sizeof(iarr[0]); i++){
      cout <<"\n address of iarr["<<i<<"] = "<< iaptr + i
      <<", value at iarr["<<i<<"] = "<< *(iaptr + i);
   }
   cout<<endl;
   double darr[5] = {10.1, 20.2, 30.3, 40.4, 50.5};
   double *daptr = darr;
   for (int i=0; i<sizeof(darr)/sizeof(darr[0]); i++){
      cout <<"\n address of darr["<<i<<"] = "<< daptr + i
      <<", value at darr["<<i<<"] = "<< *(daptr + i);
   }
}
void ArrayOfPointers(){
 int a=1, b=2, c[3]={1,2,3}, d[4]={4,5,6,7};
 int *aoptr[4];
 aoptr[0] = &a;
 aoptr[1] = &b;
```

```
 aoptr[2] = c;
 aoptr[3] = d;
 cout<<"\n value in variable a = "<<*aoptr[0];
 cout<<"\n value in variable b = "<<*aoptr[1];
 cout<<"\n value in array c = ";
 for (int i=0; i<3; i++)
   cout<<*(aoptr[2] + i)<<"\t";
 cout<<"\n value in array d = ";
 for (int i=0; i<4; i++)
   cout<<*(aoptr[3] + i)<<"\t";
 cout<<"\n adding 2 to the value of first location in array c through pointer = "<<*aoptr[2] + 1;
}
int main( )
{
//uncomment each function one by one to go through its contents.
//aliases();
//sizeOfPointers();
//assigningValuesAndDeReferencing();
//voidPointers();
//pointersAndConstants();
//PointerArithmetics_Arrays_PointerDecay();
//pointerToEntire1DArray();
//pointerToEntire2DArray();
//pointerToEntire3DArray();
//AccessingArrayWithoutDecay( );
//AccessingArrayThroughPointerDecay();
//ArrayOfPointers();
   return 0;
}
```